

# API Reference & Integration Guide

## Project: HireSense (AI-Powered Applicant Tracking System)

Document Version: 1.0.0

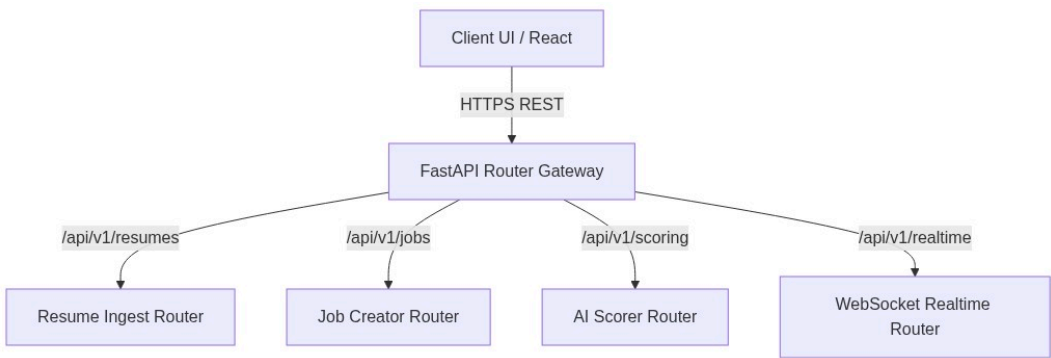
Author: Antigravity Principal Systems Engineer

Status: Approved

Target Architecture: FastAPI v0.110+ & Supabase Database Integrations

### 1. Core Endpoints Matrix

The HireSense application backend is structured as a high-performance RESTful API served by FastAPI and integrated with Supabase PostgreSQL for real-time replication.



#### 1.1 Endpoints Specification

| Method | Endpoint                  | Description                                                    | Request Header / Auth                               | Payload Format                                |
|--------|---------------------------|----------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------|
| POST   | /api/v1/resumes/upload    | Ingests, sanitizes and stores raw candidate resume PDF         | Bearer jwt_token, Content-Type: multipart/form-data | Multipart PDF binary up to 10MB               |
| GET    | /api/v1/resumes/{id}      | Retrieves parsed resume profile metadata and status            | Bearer jwt_token                                    | None                                          |
| POST   | /api/v1/jobs/create       | Posts new Job Description (JD) and triggers vector calculation | Bearer jwt_token, Content-Type: application/json    | JSON with title, raw_text, skills weights     |
| POST   | /api/v1/scoring/evaluate  | Triggers parallel AI nearest-neighbor matching query           | Bearer jwt_token, Content-Type: application/json    | JSON with resume_id, job_id, fair_mode toggle |
| GET    | /api/v1/compliance/report | Generates regulatory NYC LL144 bias metrics report             | Bearer jwt_token                                    | None                                          |
| GET    | /api/v1/scheduler/slots   | Fetches synchronized calendar availability slots               | Bearer jwt_token                                    | None                                          |

## 2. Request & Response Payload Schemas

Pydantic v2 schemas are utilized to validate and structure all API requests and responses at the network boundary.

### 2.1 Job Description Creation Schema

- Request Payload Pydantic model (JobCreateRequest):

```
from pydantic import BaseModel, Field
from typing import Dict

class JobCreateRequest(BaseModel):
    title: str = Field(..., min_length=3, max_length=100, examples=["Software Engineer (AI)"])
    job_text: str = Field(..., min_length=20, examples=["Looking for a Senior Python Developer with FastAPI experience..."])
    skills_weight: Dict[str, float] = Field(
        ...,
        examples=[{"FastAPI": 0.40, "React": 0.30, "PostgreSQL": 0.30}]
    )
```

### 2.2 Scorer Evaluation Response Schema

- Response Payload Pydantic model (EvaluationResponse):

```
from pydantic import BaseModel, UUID4, Field
from typing import Dict, Optional

class EvaluationResponse(BaseModel):
    match_id: UUID4
    resume_id: UUID4
    job_id: UUID4
    semantic_score: float = Field(..., ge=0.0, le=1.0)
    skill_score: float = Field(..., ge=0.0, le=1.0)
    experience_score: float = Field(..., ge=0.0, le=1.0)
    final_score: float = Field(..., ge=0.0, le=1.0)
    fair_mode_active: bool
    bias_report: Optional[Dict] = None
```

## 3. Supabase Real-time Trigger Integrations

To keep the recruiter's Kanban board updated with zero page refreshes, PostgreSQL triggers listen to database events and publish changes immediately to Supabase Realtime WebSocket slots.



### 3.1 PostgreSQL Trigger Configuration

This trigger function automatically broadcasts status updates when a Celery worker completes a parsing cycle:

```
-- Create trigger function to notify channel on resume update
```

```

CREATE OR REPLACE FUNCTION public.notify_resume_status_change()
RETURNS trigger AS $$
BEGIN
    -- Perform realtime broadcast notify using Supabase pg_net or standard notification channel
    PERFORM pg_notify(
        'resume_status_channel',
        json_build_object(
            'event', TG_OP,
            'resume_id', NEW.id,
            'user_id', NEW.user_id,
            'status', NEW.status,
            'ats_score', NEW.ats_score
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Bind the trigger function to the resumes table
CREATE OR REPLACE TRIGGER on_resume_status_update
AFTER UPDATE OF status ON public.resumes
FOR EACH ROW
EXECUTE FUNCTION public.notify_resume_status_change();

```

This configuration enables sub-second interface state routing with zero application layer polling!